

PROJECT 3 REPORT: SQL DATA ANALYSIS

Name: Favour Secondus

Batch: July, 2026.

PROBLEM STATEMENT

To analyze customer order data using SQL to identify top-performing products, monthly revenue trends, order status distribution, and outliers (high value customers).

METHODOLOGY

Data was imported into PostgreSQL from a cleaned CSV dataset containing 1200 orders and 1400 columns. SQL queries were written using:

SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, Aggregate functions: SUM(), AVG(), COUNT(), and EXTRACT() for month-based analysis.

Outliers- orders above the already calculated upper bound (3333.53) were identified.

KEY FINDINGS

METRIC	RESULT
Total Revenue	1,264,743
Average Order Value	1,053.96
Total Orders	1,200
Top Product	Chair (195,622)
Highest Revenue Month	June (170,611)
Highest Realized Revenue Month (delivered only)	June (38,799)
Least Revenue Month	September (69,323)
Least Realized Revenue Month (delivered only)	February (7,592)
Revenue Gap (June)	131,812 (lost to cancellations)
Most Common Order Status	Cancelled (250 orders)
Delivered Product	Laptop (40,715)
VIP Orders (3,333.53)	8 customers

SQL QUERIES USED

Query 1: Filtered Orders (WHERE)

DECODELAB'S PRO... x deodelabs/postgres@PostgreSQL 18* x

deodelabs/postgres@PostgreSQL 18

Query Query History

```
1 SELECT *
2 FROM deodelabs
3 WHERE totalprice > 1000
4 ORDER BY totalprice DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 512 Page No: 1 of 1

	orderid [PK] character varying (15)	date date	customerid character varying (15)	product character varying (15)	quantity integer	unitprice numeric (15)	shippingaddress character varying (15)	paymentmethod character varying (15)	ordersta character
1	ORD200789	2023-08-...	C57276	Tablet	5	691	183 Main St	Online	Delivere
2	ORD200632	2023-05-...	C67260	Laptop	5	678	463 Main St	Gift Card	Delivere
3	ORD201122	2023-06-...	C38840	Monitor	5	678	766 Main St	Online	Returne
4	ORD200469	2023-11-...	C13877	Chair	5	677	893 Main St	Cash	Cancelle
5	ORD200328	2023-02-...	C18404	Tablet	5	674	546 Main St	Online	Cancelle
6	ORD200107	2023-03-...	C16775	Printer	5	671	848 Main St	Gift Card	Shipped
7	ORD200326	2024-07-...	C65986	Laptop	5	670	273 Main St	Gift Card	Returne
8	ORD201065	2023-10-...	C47778	Printer	5	667	488 Main St	Debit Card	Delivere
9	ORD201031	2023-02-...	C59183	Phone	5	665	136 Main St	Debit Card	Pending
10	ORD200463	2023-05-...	C25276	Laptop	5	663	214 Main St	Debit Card	Shipped
11	ORD200361	2024-06-...	C53464	Printer	5	660	197 Main St	Credit Card	Delivere
12	ORD200367	2024-04-...	C13108	Laptop	5	659	712 Main St	Gift Card	Pending
13	ORD200837	2025-06-...	C88020	Chair	5	656	282 Main St	Online	Delivere

Total rows: 512 Query complete 00:00:00.222 CRLF Ln 4, Col 26

Query 2: Revenue by Product (GROUP BY + SUM)

DECODELAB'S PRO... x deodelabs/postgres@PostgreSQL 18* x

deodelabs/postgres@PostgreSQL 18

Query Query History

```
4 ORDER BY totalprice DESC;
5 SELECT product,
6 SUM (totalprice) AS total_revenue
7 FROM deodelabs
8 GROUP BY product
9 ORDER BY total_revenue DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	product character varying (15)	total_revenue numeric
1	Chair	195622
2	Printer	195610
3	Laptop	192122
4	Tablet	186570
5	Monitor	175648
6	Desk	167458
7	Phone	151718

Total rows: 7 Query complete 00:00:00.218 CRLF Ln 9, Col 29

Query 3: Revenue by Month (EXTRACT + GROUP BY)

Help

DECODELAB'S PRO... X decodelabs/postgres@PostgreSQL 18* X

decodelabs/postgres@PostgreSQL 18

Query Query History

```
10 SELECT
11 EXTRACT (MONTH FROM date) AS month,
12 SUM (totalprice) AS total_revenue
13 FROM decodelabs
14 GROUP BY month
15 ORDER BY total_revenue DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	month numeric	total_revenue numeric
1	6	170611
2	5	135139
3	1	124315
4	3	123841
5	2	112349
6	4	109183
7	10	89833
8	8	86342
9	7	85780
10	12	82541
11	11	75491
12	9	68979

Total rows: 12 Query complete 00:00:00.203 CRLF Ln 15, Col 29

Query 4: Realized Revenue by Month for Delivered Products (EXTRACT + GROUP BY + WHERE)

Help

DECODELAB'S PRO... X decodelabs/postgres@PostgreSQL 18* X

decodelabs/postgres@PostgreSQL 18

Query Query History

```
56 SUM (totalprice) AS realized_revenue
57 FROM decodelabs
58 WHERE orderstatus = 'Delivered'
59 GROUP BY month
60 ORDER BY realized_revenue DESC;
61
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	month numeric	realized_revenue numeric
1	6	38799
2	10	32900
3	5	28510
4	7	22953
5	4	22477
6	1	20285
7	3	19467
8	8	16218
9	11	14773
10	12	10459
11	9	8165
12	2	7500

Total rows: 12 Query complete 00:00:00.084 CRLF Ln 60, Col 29

Query 5: Month with the Least Revenue (EXTRACT + GROUP BY + LIMIT 5)

Help

DECODELAB'S PRO... X deodelabs/postgres@PostgreSQL 18* X

deodelabs/postgres@PostgreSQL 18

Query Query History

```
16 SELECT
17 EXTRACT (MONTH FROM date) AS month,
18 SUM (totalprice) AS total_revenue
19 FROM deodelabs
20 GROUP BY month
21 ORDER BY total_revenue ASC LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	month numeric	total_revenue numeric
1	9	69323
2	11	75491
3	12	82541
4	7	85780
5	8	86342

Total rows: 5 Query complete 00:00:00.390 CRLF Ln 21, Col 35

Query 6: Order Status Counts (GROUP BY + COUNT)

Help

DECODELAB'S PRO... X deodelabs/postgres@PostgreSQL 18* X

deodelabs/postgres@PostgreSQL 18

Query Query History

```
11 EXTRACT (MONTH FROM date) AS month,
12 SUM (totalprice) AS total_revenue
13 FROM deodelabs
14 GROUP BY month
15 ORDER BY total_revenue DESC;
16 SELECT orderstatus,
17 COUNT (*) AS total_orders
18 FROM deodelabs
19 GROUP BY orderstatus
20 ORDER BY total_orders DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	orderstatus character varying (15)	total_orders bigint
1	Cancelled	250
2	Returned	247
3	Pending	237
4	Shipped	235
5	Delivered	231

Total rows: 5 Query complete 00:00:00.161 CRLF Ln 16, Col 20

Query 7: Aggregations

Help

DECODELAB'S PRO... x decodelabs/postgres@PostgreSQL 18* x

decodelabs/postgres@PostgreSQL 18

Query Query History

```
30 SELECT
31     product,
32     SUM (totalprice) AS total_revenue
33 FROM decodelabs
34 GROUP BY product
35 HAVING SUM(totalprice) > 150000
36 ORDER BY total_revenue DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	product character varying (15)	total_revenue numeric
1	Chair	195622
2	Printer	195610
3	Laptop	192122
4	Tablet	186570
5	Monitor	175648
6	Desk	167458
7	Phone	151718

Total rows: 7 Query complete 00:00:00.141 CRLF Ln 36, Col 29

Query 8: Products above 150,000 (HAVING

Help

DECODELAB'S PRO... x decodelabs/postgres@PostgreSQL 18* x

decodelabs/postgres@PostgreSQL 18

Query Query History

```
19 GROUP BY orderstatus
20 ORDER BY total_orders DESC;
21 SELECT
22     SUM (totalprice) AS total_revenue,
23     AVG (totalprice) AS avg_order_value
24 FROM decodelabs;
25 SELECT
26     SUM (totalprice) AS total_revenue,
27     AVG (totalprice) AS avg_order_value,
28     COUNT(*) AS total_orders
29 FROM decodelabs;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	total_revenue numeric	avg_order_value numeric	totalOrders bigint
1	1264748	1053.956666666666667	1200

Total rows: 1 Query complete 00:00:00.211 CRLF Ln 29, Col 17

Query 9: Top 3 Delivered products (WHERE + LIMIT)

Help

DECODELAB'S PRO... X deodelabs/postgres@PostgreSQL 18* X

deodelabs/postgres@PostgreSQL 18

Query Query History

```
37 SELECT
38     product,
39     SUM (totalprice) AS total_revenue
40 FROM deodelabs
41 WHERE orderstatus = 'Delivered'
42 GROUP BY product
43 ORDER BY total_revenue DESC LIMIT 3;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	product character varying (15)	total_revenue numeric
1	Laptop	40715
2	Phone	40342
3	Printer	38052

Total rows: 3 Query complete 00:00:00.129 CRLF Ln 43, Col 37

Query 10: Outliers (VIP Customers)

Help

DECODELAB'S PRO... X deodelabs/postgres@PostgreSQL 18* X

deodelabs/postgres@PostgreSQL 18

Query Query History

```
44 SELECT *
45 FROM deodelabs
46 WHERE totalprice > 3333.53
47 ORDER BY totalprice DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	orderid [PK] character varying (15)	date date	customerid character varying (15)	product character varying (15)	quantity integer	unitprice numeric (15)	shippingaddress character varying (15)	paymentmethod character varying (15)	orderstatus character varying (15)
1	ORD200789	2023-08-...	C57276	Tablet	5	691	183 Main St	Online	Delivered
2	ORD201122	2023-06-...	C38840	Monitor	5	678	766 Main St	Online	Returned
3	ORD200632	2023-05-...	C67260	Laptop	5	678	463 Main St	Gift Card	Delivered
4	ORD200469	2023-11-...	C13877	Chair	5	677	893 Main St	Cash	Cancelled
5	ORD200328	2023-02-...	C18404	Tablet	5	674	546 Main St	Online	Cancelled
6	ORD200107	2023-03-...	C16775	Printer	5	671	848 Main St	Gift Card	Shipped
7	ORD200326	2024-07-...	C65986	Laptop	5	670	273 Main St	Gift Card	Returned
8	ORD201065	2023-10-...	C47778	Printer	5	667	488 Main St	Debit Card	Delivered

Total rows: 8 Query complete 00:00:00.104 CRLF Ln 47, Col 26

Query 11: Month with the Most Cancellations

The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
61 SELECT
62     EXTRACT (MONTH FROM date) AS month,
63     COUNT (*) AS cancelled_orders
64 FROM decode_labs
65 WHERE orderstatus = 'Cancelled'
66 GROUP BY month
67 ORDER BY cancelled_orders DESC;
```

The 'Data Output' tab shows the results of the query, displaying 12 rows. The columns are 'month' (numeric) and 'cancelled_orders' (bigint). The results are ordered by the number of cancellations in descending order.

	month numeric	cancelled_orders bigint
1	6	34
2	4	30
3	3	29
4	5	25
5	2	24
6	8	21
7	1	20
8	7	17
9	11	15
10	10	13
11	12	11

At the bottom of the interface, it shows 'Total rows: 12', 'Query complete 00:00:00.120', and the cursor position 'Ln 67, Col 32'.

RECOMMENDATIONS

1. **Prioritize Top-Performing Products:** Chair (195,622), Printer (195,610), Laptop (192,122), and Tablet (186,570) generate the highest revenue. These products should be prioritized in marketing campaigns, inventory planning, and sales promotions.
2. **Investigate High Cancellation Rate:** Cancelled is the most common order status, accounting for 250 orders. A root cause analysis is recommended to identify whether cancellations are driven by stock unavailability, delivery delays, pricing dissatisfaction, or customer behavior patterns.
3. **Optimize Marketing for Slow Months:** February, September, and December are the 3 least performing revenue months. Seasonal promotions, targeted email campaigns, and discount strategies should be deployed during slow months to maintain consistent revenue throughout the year. October is the second-highest month for realized revenue. Identify what worked in June and October and replicate in these months.

4. Strengthen Delivered Product Strategy: Laptop is the revenue generating product among delivered orders (40,715). Ensuring consistent stock availability, product quality, and fast delivery for Laptops can further boost customer satisfaction and revenue.
5. June drives the highest overall revenue (170,611) but also has the most cancellations (34). Inventory, shipping, and customer support capacity should be reviewed.

CONCLUSION

This project successfully demonstrated the application of SQL to derive meaningful business insights from a customer order database. A total of 1200 orders were analyzed using structured queries to identify revenue drivers, seasonal patterns, top performing products, order behavior, and high value customers. The analysis revealed **three** clear priorities.

Firstly, June drives the highest gross revenue but also the highest cancellation volume, (170,611 in orders versus 38,799 delivered) indicating a fulfillment gap during peak demand that requires operational review. Fixing June operations would directly improve realized revenue.

Initially, September generated the least revenue but when filtered to delivered orders only, February emerges as the weakest month, shifting the focus from cancellations to demand generation.

Third, eight customers exceed the 3,333 spending threshold, representing a high-value segment that warrants targeted retention.

These findings confirm that SQL is an effective tool for isolating, verifying, and revisiting business metrics. The queries developed in this project can be reused for ongoing monitoring and adapted as new questions emerge.